# Llama Inference C++: Full Forward Pass Handbook

Entry Point to Token Generation (with Color Diagrams)

llama_inference_cpp

March 5, 2026

## Contents


**1 Purpose** **2**

**2 Code map (where each step lives)** **2**

**3 Color Legend** **2**

**4 Diagram 1: End-to-end runtime path** **3**

**5 Entry point block-by-block** **3**
- 5.1 1) CLI parsing in `main.cpp` . . . 3
- 5.2 2) Model load path in `engine_lifecycle.cpp` . . . 4
- 5.3 3) Tokenizer load/encode/decode in `tokenizer.cpp` . . . 4

**6 Diagram 2: Forward pass compute graph (single decode step)** **5**

**7 Forward pass block explanations in code order** **6**
- 7.1 A) Preparation . . . 6
- 7.2 B) Embedding boundary . . . 6
- 7.3 C) Attention block per layer . . . 6
- 7.4 D) FFN block per layer . . . 6
- 7.5 E) Final logits . . . 6

**8 Sampling and generation path** **6**

**9 Function reference (quick lookup)** **7**

**10 Practical reading order** **7**

**11 Line-by-line annotated walkthrough** **8**
- 11.1 `src/main.cpp` (program entry) . . . 8
- 11.2 `src/engine_lifecycle.cpp` (load and memory setup) . . . 8
- 11.3 `src/tokenizer.cpp` (prompt to token IDs and back) . . . 9
- 11.4 `src/engine_math_kernels.cpp` (primitive compute blocks) . . . 9
- 11.5 `src/engine_forward.cpp` (single-token transformer forward) . . . 10
- 11.6 `src/engine_sampling.cpp` (logits to next token) . . . 10

# 1 Purpose

This handbook explains the full runtime path in the current C++ codebase:

- from CLI entry point,
- through model loading and forward-pass blocks,
- to sampling and final token output.

It is structured block-by-block, with diagrams and file/function mapping.

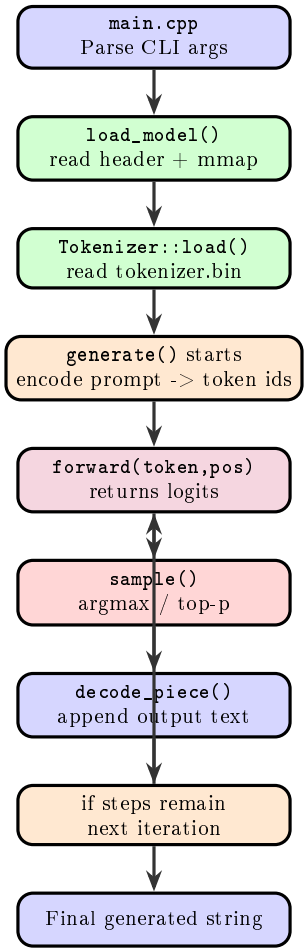
# 2 Code map (where each step lives)

| File | Responsibility |
|---|---|
| `src/main.cpp` | CLI entry point, argument parsing, invokes model/tokenizer load and generation. |
| `src/engine_lifecycle.cpp` | Constructor/destructor, checkpoint mmap, weight pointer mapping, run-state allocation. |
| `src/tokenizer.cpp` | Tokenizer binary loading, encode (BPE), decode piece-by-piece. |
| `src/engine_math_kernels.cpp` | Primitive compute blocks: RMSNorm, softmax, matmul (GEMV style). |
| `src/engine_forward.cpp` | Full transformer forward pass for one token at one position. |
| `src/engine_sampling.cpp` | Sampling methods (argmax, multinomial, top-p) and autoregressive generation loop. |
| `include/llama_engine.hpp` | Public class/struct declarations and private engine internals. |

# 3 Color Legend

- Blue: I/O boundaries (token input, output text)
- Green: normalization and stabilizing operations
- Orange: linear projections/matmul blocks
- Purple: attention-specific operations
- Red: nonlinear or sampling decisions

# 4 Diagram 1: End-to-end runtime path




# 5 Entry point block-by-block

## 5.1 1) CLI parsing in `main.cpp`

The executable reads:

- checkpoint path,
- tokenizer path,
- prompt,
- generation controls: `steps`, `temperature`, `topp`, `seed`.

Then:

```
LlamaEngine engine;
engine.load_model(checkpoint);
Tokenizer tokenizer;
tokenizer.load(tokenizer_path, engine.config().vocab_size);
std::string text = engine.generate(tokenizer, prompt, steps, temperature, topp, seed);
```

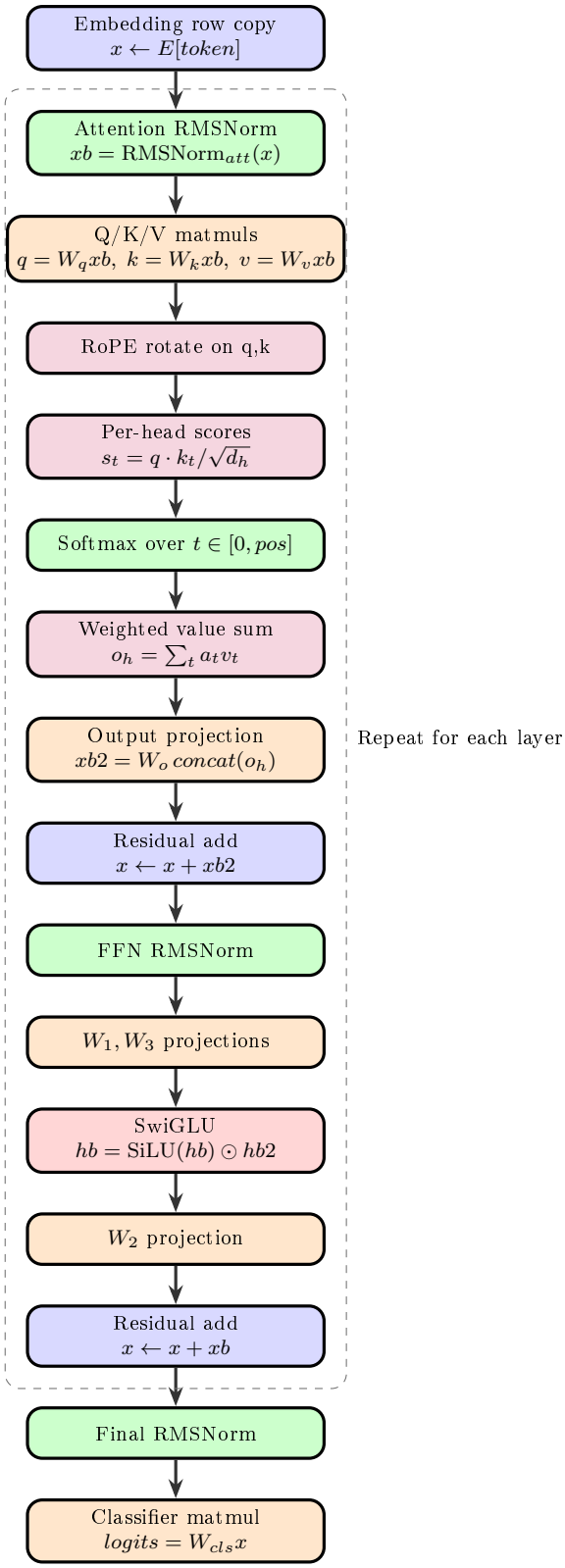
### 5.2 2) Model load path in `engine_lifecycle.cpp`

1. open checkpoint file,
2. read `Config`,
3. map file into memory ( `mmap`),
4. compute weight pointer offsets ( `memory_map_weights`),
5. allocate run-state tensors and KV caches.

### 5.3 3) Tokenizer load/encode/decode in `tokenizer.cpp`

- `load()`: reads token strings and scores from tokenizer binary.
- `encode()`: BOS prefix, UTF-8 byte handling, byte fallback, iterative BPE merges.
- `decode_piece()`: token-id to printable piece conversion.

## 6 Diagram 2: Forward pass compute graph (single decode step)



# 7 Forward pass block explanations in code order

## 7.1 A) Preparation

In `engine_forward.cpp`, the function caches local dimensions: `dim, kv_dim, kv_mul, hidden_dim, head_size`.

## 7.2 B) Embedding boundary

It copies a single embedding row into `x`. This is the starting representation for current token position.

## 7.3 C) Attention block per layer

1. attention RMSNorm,
2. Q/K/V matmuls,
3. RoPE rotation for q/k channel pairs,
4. score computation over cached timesteps,
5. softmax to attention probabilities,
6. weighted sum over value cache,
7. output projection and residual add.

## 7.4 D) FFN block per layer

1. FFN RMSNorm,
2. $W_1$ and $W_3$ projections,
3. SwiGLU gating,
4. $W_2$ projection,
5. residual add.

## 7.5 E) Final logits

After all layers:

- final RMSNorm,
- classifier matmul to produce logits for next token.

# 8 Sampling and generation path

In `engine_sampling.cpp`:

- If `temperature==0`, choose argmax (deterministic).
- Else scale logits by temperature, softmax, then sample:
  - full multinomial if top-p disabled,
  - nucleus (top-p) truncation + sample otherwise.
- Decode selected token piece and append to output text.

## 9 Function reference (quick lookup)

| Function | Purpose |
|---|---|
| `load_model()` | Reads config, mmaps checkpoint, maps weight pointers, allocates run state. |
| `Tokenizer::load()` | Loads tokenizer vocabulary and scores. |
| `Tokenizer::encode()` | Prompt text to token ids. |
| `forward(token,pos)` | One full transformer step returning logits. |
| `sample(...)` | Chooses next token id from logits distribution. |
| `generate(...)` | Full autoregressive loop from prompt to final text. |

## 10 Practical reading order

Recommended order if studying implementation deeply:

1. `src/main.cpp`
2. `src/engine_lifecycle.cpp`
3. `src/tokenizer.cpp`
4. `src/engine_forward.cpp`
5. `src/engine_sampling.cpp`
6. `src/engine_math_kernels.cpp`

# 11 Line-by-line annotated walkthrough

This section gives a direct code explanation from entry point to final token output. Line numbers below match the current repository files.

## 11.1 `src/main.cpp` (program entry)

| Line(s) | Explanation |
|---|---|
| 1 | Pulls in public API declarations (`LlamaEngine`, `Tokenizer`, `Config`). |
| 3-6 | Standard libraries for argument parsing, time seed, and terminal output. |
| 8 | Uses the project namespace so symbols can be referenced directly. |
| 10-19 | Defines `usage()` to print supported CLI flags and defaults. |
| 21 | Process entrypoint receives raw C arguments. |
| 22-25 | Enforces required checkpoint argument; exits with code 1 on invalid call. |
| 27-33 | Initializes defaults: tokenizer path, empty prompt, steps, temperature, top-p, seed. |
| 35-47 | Parses optional flags with sequential scan and updates local settings. |
| 49 | Starts exception boundary for runtime failures (I/O, malformed files, etc.). |
| 50-51 | Creates engine and loads model checkpoint into mapped memory. |
| 53-54 | Creates tokenizer and loads tokenizer vocabulary with model vocab size. |
| 56 | Runs full prompt-to-generation flow and returns generated text string. |
| 57 | Prints final generated text to stdout. |
| 58-61 | Converts thrown exceptions to readable error output and exit code 2. |
| 63 | Successful process exit. |

## 11.2 `src/engine_lifecycle.cpp` (load and memory setup)

| Line(s) | Explanation |
|---|---|
| 12 | Default constructor; actual state is initialized lazily in `load_model`. |
| 14-22 | Destructor unmaps checkpoint, closes file descriptor, and frees run-state allocations. |
| 24-25 | Starts run-state allocation and computes KV dimension per token position. |
| 27-39 | Allocates all working buffers (`x`, `xb`, `hb`, `q`, cache, attention, logits). |
| 33-35 | Keeps `k` and `v` as aliases (not owned memory), avoiding invalid free. |
| 41-44 | Hard-fails if any allocation returns null. |
| 47-60 | Frees owned buffers and sets pointers to null to avoid use-after-free or double free. |
| 62-92 | Walks contiguous checkpoint tensor layout and assigns each `weights_.*` pointer. |
| 91 | Sets classifier weights to tied embeddings when shared-weights flag is enabled. |
| 94-98 | Opens checkpoint header stream and throws with context if missing. |
| 100-103 | Reads `Config` struct from file header. |
| 105-107 | Decodes legacy sign convention for tied embeddings and normalizes vocab size. |
| 109-111 | Gets total file size for later `mmap`. |
| 113-121 | Opens FD and memory maps model file into process address space. |
| 123-125 | Offsets past config header and maps all weight pointers. |

| Line(s) | Explanation |
|---|---|
| 126 | Allocates runtime activations and KV cache now that config is known. |

### 11.3 `src/tokenizer.cpp` (prompt to token IDs and back)

| Line(s) | Explanation |
|---|---|
| 11-13 | `qsort`/`bsearch` comparator for token string lexicographic order. |
| 15-20 | Initializes tokenizer containers and invalidates sorted lookup cache. |
| 22-25 | Precomputes one-byte printable lookup pieces for raw-byte fallback decoding. |
| 27-30 | Opens tokenizer binary and throws on failure. |
| 32-35 | Reads maximum token length metadata from tokenizer file. |
| 38-57 | For each vocab entry: read score, read byte length, allocate token bytes, null-terminate. |
| 62-71 | Lazily builds sorted vocabulary index the first time lookup is needed. |
| 73-77 | Binary-searches token string and returns token id or `-1`. |
| 80-83 | Starts encode path and reserves token vector capacity. |
| 84-86 | Optional BOS token insertion. |
| 88-91 | Adds dummy-prefix token for sentencepiece-style leading-space behavior. |
| 93-96 | Creates temporary UTF-8 assembly buffer for candidate piece lookup. |
| 97-123 | UTF-8 aware scan: assemble symbol, lookup piece, fallback to raw bytes if unknown. |
| 125-146 | Greedy BPE merge loop: repeatedly merge best-scoring adjacent pair until convergence. |
| 148-150 | Optional EOS token append. |
| 152 | Returns final prompt token sequence. |
| 155-159 | Decoding helper trims leading space after BOS continuation. |
| 161-164 | Converts byte-escape tokens like `<0xAB>` into real single-byte pieces. |
| 166-175 | Filters empty or non-printable single-byte fragments for cleaner terminal output. |
| 177 | Returns human-readable token piece string. |

### 11.4 `src/engine_math_kernels.cpp` (primitive compute blocks)

| Line(s) | Explanation |
|---|---|
| 7 | `rmsnorm`: normalization primitive used before attention and FFN blocks. |
| 9-12 | Computes sum of squared activations. |
| 13-15 | Converts to inverse RMS with epsilon for numerical stability. |
| 16-18 | Scales normalized input by learned per-channel weights. |
| 21 | `softmax`: converts logits/scores into probability distribution. |
| 23-28 | Finds max value to stabilize exponentials. |
| 30-34 | Exponentiates shifted values and accumulates denominator sum. |
| 35-37 | Normalizes every entry by denominator. |
| 40 | `matmul`: reference GEMV kernel used for linear projections. |
| 42-48 | For each output row, dot product row weights with input vector. |

### 11.5 `src/engine_forward.cpp` (single-token transformer forward)

| Line(s) | Explanation |
|---|---|
| 8 | Starts one-step decode forward: input is current token id and position. |
| 9-14 | Caches frequently used dimensions (`dim`, KV width, FFN hidden size, head size). |
| 16-18 | Copies token embedding row into working state vector `x`. |
| 20 | Iterates all transformer layers sequentially. |
| 22 | Applies attention RMSNorm to current residual stream. |
| 24-27 | Computes layer+position offset and binds `k`/`v` aliases into KV cache row. |
| 28-30 | Computes Q, K, V linear projections. |
| 33-47 | Applies RoPE rotation pairwise to query, and to key where KV dimension covers it. |
| 50-52 | Loops each attention head and selects per-head query and attention buffer slice. |
| 54-62 | Computes causal attention scores against all timesteps `[0..pos]`. |
| 64 | Softmax converts raw scores to attention probabilities. |
| 66-74 | Computes weighted sum of cached values for this head. |
| 78 | Projects concatenated head outputs with `W_o`. |
| 79-81 | First residual connection: adds attention output into `x`. |
| 84 | Applies FFN RMSNorm. |
| 86-87 | Computes two FFN branches `W1(x)` and `W3(x)`. |
| 90-95 | Applies SwiGLU gate: `silu(W1(x)) * W3(x)` element-wise. |
| 97 | Projects gated hidden vector through `W2` back to model dimension. |
| 98-100 | Second residual connection: FFN output added back into `x`. |
| 104 | Applies final RMSNorm after last layer. |
| 105 | Computes logits over full vocabulary with classifier matrix. |
| 107 | Returns logits as `std::vector<float>`. |

### 11.6 `src/engine_sampling.cpp` (logits to next token)

| Line(s) | Explanation |
|---|---|
| 8-18 | `sample_argmax`: deterministic next-token selection (maximum probability). |
| 20-29 | `sample_mult`: cumulative CDF walk for multinomial sampling. |
| 31-37 | Comparator for sorting probabilities descending in top-p sampling. |
| 39-49 | Top-p prefilter: remove entries below nucleus cutoff floor. |
| 51 | Sorts surviving candidates by probability descending. |
| 53-61 | Finds smallest prefix whose cumulative mass exceeds `topp`. |
| 63-71 | Samples inside nucleus prefix via CDF and mapped random coin. |
| 74-79 | Xorshift-based integer RNG step for deterministic reproducibility. |
| 81-83 | Converts RNG bits into float in `[0,1)`. |
| 85-89 | `sample`: greedy path when temperature is exactly zero. |
| 91-94 | Temperature scales logits, then softmax converts to probabilities. |
| 96-100 | Chooses multinomial or top-p strategy based on `topp` range. |
| 103-110 | `generate` signature: prompt text to generated output string. |
| 112-115 | Tokenizes prompt and guards against empty token sequence. |
| 117-118 | Allocates reusable top-p scratch buffer and initializes RNG state. |

| Line(s) | Explanation |
|---|---|
| `120-126` | Initializes autoregressive loop state and output string buffer. |
| `127-130` | Each step runs forward pass to get logits for next token. |
| `132-136` | Forces prompt tokens while prompt remains; samples afterward. |
| `137` | Advances decode position. |
| `139-141` | Stops on EOS token id 1 (project convention used by this checkpoint path). |
| `143-145` | Decodes token piece, appends text, then shifts context token. |
| `148` | Returns final generated text. |